

Code Complexity Analysis for USEI15

Directed Line Upgrade Plan

Project: Logistics On Rails

Sprint: 3

User Story: USEI15 - Risk-Aware Shortest Paths **Date:** December 2024

Overview

This document analyzes the time complexity of all critical methods in the USEI11 implementation using Big-O notation.

Dataset: Belgian Railway Network

- $V = 559$ stations (vertices)
- $E = 691$ connections (edges)

Algorithms:

- Colored DFS (Cycle Detection) - ESINF06-Graph.pdf, slides 90-92
 - Kahn's Algorithm (Topological Sort) - ESINF06-Graph.pdf, slide 116
-

Method Analysis

1. `BellmanFordShortestPath.shortestPath()`

Code:

```

public ShortestPathResult<V> shortestPath(
    Graph<V, E> graph,
    V source,
    V target,
    ToDoubleFunction<E> costFunction
) {
    int n = graph.numVertices(); // O(1)
    double[] dist = new double[n]; // O(V)
    int[] parent = new int[n]; // O(V)

    Arrays.fill(dist, Double.POSITIVE_INFINITY); // O(V)
    Arrays.fill(parent, -1); // O(V)

    int srcKey = graph.key(source); // O(1)
    dist[srcKey] = 0; // O(1)

    // Relax edges V-1 times
    for (int i = 1; i < n; i++) { // O(V)
        for (Edge<V, E> e : graph.edges()) { // O(E)
            int u = graph.key(e.getVOrig()); // O(1)
            int v = graph.key(e.getVDest()); // O(1)
            double w = costFunction.applyAsDouble(
                e.getWeight()); // O(1)

            if (dist[u] != Double.POSITIVE_INFINITY
                && dist[u] + w < dist[v]) { // O(1)
                dist[v] = dist[u] + w; // O(1)
                parent[v] = u; // O(1)
            }
        }
    }

    // Total Loop: O(V * E)

    // Detect negative cycles
    for (Edge<V, E> e : graph.edges()) { // O(E)
        int u = graph.key(e.getVOrig());
        int v = graph.key(e.getVDest());
        double w = costFunction.applyAsDouble(
            e.getWeight()); // O(1)

        if (dist[u] != Double.POSITIVE_INFINITY
            && dist[u] + w < dist[v]) { // O(1)

            // 1. Extract cycle vertices

```

```

        List<V> cycle = extractCycle(graph, parent, v); // O(V)

        // 2. Generate detailed report
        String report = generateCycleReport(
            graph, cycle, costFunction); // O(V)

        // 3. Throw exception
        throw new NegativeCycleException(cycle, report); // O(1)
    }
}

// Check target reachability
int targetKey = graph.key(target); // O(1)
if (dist[targetKey] == Double.POSITIVE_INFINITY) { // O(1)
    return ShortestPathResult.noPath(); // O(1)
}

// Reconstruct path
List<V> path = new ArrayList<>(); // O(1)
int current = graph.key(target); // O(1)

while (current != -1) { // O(V)
    path.add(graph.vertex(current)); // O(1)
    current = parent[current]; // O(1)
}

Collections.reverse(path); // O(V)

// Build cost map
Map<V, Double> costMap = new HashMap<>(); // O(1)
for (int i = 0; i < dist.length; i++) { // O(V)
    costMap.put(graph.vertex(i), dist[i]); // O(1)
}

return new ShortestPathResult<>(
    path, costMap, dist[targetKey], true); // O(1)
}

```

Overall Complexity: $O(V * E)$

Deterministic: YES

2. BellmanFordShortestPath.extractCycle()

Code:

```
private List<V> extractCycle(Graph<V, ?> graph, int[] parent, int start) {
    Set<Integer> visited = new HashSet<>(); // O(1)
    int v = start;

    // Find cycle start by backtracking
    while (!visited.contains(v)) { // O(V) worst case
        visited.add(v); // O(1)
        v = parent[v]; // O(1)
    }

    int cycleStart = v;
    List<V> cycle = new ArrayList<>(); // O(1)
    cycle.add(graph.vertex(cycleStart)); // O(1)

    v = parent[cycleStart];

    // Reconstruct the full cycle
    while (v != cycleStart) { // O(V) worst case
        cycle.add(graph.vertex(v)); // O(1)
        v = parent[v]; // O(1)
    }

    cycle.add(graph.vertex(cycleStart)); // O(1)
    Collections.reverse(cycle); // O(V)

    return cycle; // O(1)
}
```

Overall Complexity: $O(V)$

Deterministic: YES

3. BellmanFordShortestPath.generateCycleReport()

Code:

```

private String generateCycleReport(Graph<V, E> graph, List<V> cycle,
                                ToDoubleFunction<E> costFunction) {
    StringBuilder sb = new StringBuilder();           // O(1)
    double totalCost = 0;                             // O(1)

    for (int i = 0; i < cycle.size() - 1; i++) {      // O(V)
        V u = cycle.get(i);                           // O(1)
        V v = cycle.get(i + 1);                       // O(1)

        Edge<V, E> edge = graph.edge(u, v);           // O(1)

        double cost = 0;
        if (edge != null) {
            cost = costFunction.applyAsDouble(
                edge.getWeight());                     // O(1)
        }
        totalCost += cost;                             // O(1)

        // String formatting operations
        sb.append(String.format(...));                 // O(1)
    }

    return sb.toString();                             // O(V)
}

```

Overall Complexity: $O(V)$

Deterministic: YES

4. USEI15Controller.execute()

Code:

```

public void execute(Station source, Station target) {
    BellmanFordShortestPath<Station, Connection> algorithm =
        new BellmanFordShortestPath<>(); // O(1)

    try {
        ShortestPathResult<Station> result = algorithm.shortestPath(
            graph, source, target, Connection::getCost); // O(V * E)

        printShortestPath(result); // O(V)

    } catch (NegativeCycleException e) {
        System.out.println("\n=== Negative Cycle Detected ==="); // O(1)
        System.out.println(e.getDetailedMessage()); // O(1)
    }
}

```

Overall Complexity: $O(V * E)$

Deterministic: YES

5. USEI15Controller.printShortestPath()

Code:

```

private void printShortestPath(ShortestPathResult<Station> result) {
    if (!result.hasPath()) { // O(1)
        System.out.println("No path exists."); // O(1)
        return;
    }

    List<Station> path = result.getPath(); // O(1)

    for (Station s : path) { // O(V)
        System.out.printf(" - %s (%s) [cost: %.2f]\n", // O(1)
            s.getId(),
            s.getName(),
            result.getCostTo(s)); // O(1)
    }

    System.out.printf("Total cost: %.2f\n",
        result.getTotalCost()); // O(1)
}

```

Overall Complexity: $O(V)$

Deterministic: YES

6. ShortestPathResult (DTO Methods)

Code:

```
public class ShortestPathResult<V> {

    // Constructor
    public ShortestPathResult(List<V> path, Map<V, Double> costToVertex,
                               double totalCost, boolean hasPath) {

        this.path = path;                                // O(1)
        this.costToVertex = costToVertex;                // O(1)
        this.totalCost = totalCost;                      // O(1)
        this.hasPath = hasPath;                          // O(1)
    }

    public List<V> getPath() {

        return path;                                     // O(1)
    }

    public double getCostTo(V vertex) {

        return costToVertex.getOrDefault(
            vertex, Double.POSITIVE_INFINITY);          // O(1)
    }

    public static <V> ShortestPathResult<V> noPath() {

        return new ShortestPathResult<> (
            Collections.emptyList(),                     // O(1)
            Collections.emptyMap(),                      // O(1)
            Double.POSITIVE_INFINITY,
            false);                                       // O(1)
    }
}
```

Overall Complexity: $O(1)$

Deterministic: YES

Summary Table

Method	Class	Time Complexity	Deterministic
<code>shortestPath()</code>	BellmanFordShortestPath	$O(V * E)$	YES
<code>extractCycle()</code>	BellmanFordShortestPath	$O(V)$	YES
<code>generateCycleReport()</code>	BellmanFordShortestPath	$O(V)$	YES
<code>execute()</code>	USEI15Controller	$O(V * E)$	YES
<code>printShortestPath()</code>	USEI15Controller	$O(V)$	YES
<code>ShortestPathResult</code>	ShortestPathResult	$O(1)$	YES

Where:

- $V = 559$ stations (vertices)
- $E = 691$ connections (edges)

Global Complexity

Overall USEI15 Complexity: $O(V * E)$

Breakdown:

1. Initialization: $O(V)$
2. Relaxation Loop: $O(V * E)$
3. Negative Cycle Check: $O(E)$
4. Path Reconstruction: $O(V)$
5. Result Processing: $O(V)$

Dominant Term: $O(V * E)$ from the Bellman-Ford main relaxation loop.

Belgian Railway Dataset Performance:

- $V = 559, E = 691$
- The algorithm is highly efficient for this dataset size, typically executing in milliseconds.

Determinism

Is USEI15 Deterministic? YES

All methods are deterministic:

- Given the same graph and input stations, the algorithm always produces the exact same path and cost.
- Iteration order over edges is consistent (assuming Graph implementation uses ordered collections like LinkedHashMap or Lists, which is standard).
- No random number generation or external dependencies.

